PRODUCT & IMPLEMENTATION PLAN

Homework Helper

Product and Implementation Plan

Phase A Website Delivery and Mobile Handoff Update

WEBSITE STATUS	Website Phase A delivered on branch firstUpdate

Homework Helper
Updated plan - August 17, 2026

Contents

Revision at a Glance 4

- What was completed . . . 4
- What still needs to happen . . . 5

Product and UX 6

- Summary . . . 6
- Core navigation . . . 6
- Primary student journey . . . 6

Phase A - Website Implementation 6

- Delivered personal-server experience . . . 6
- Identity and student profile . . . 6
- Courses, assignments, and commitments . . . 7
- Capture and confirmation . . . 7
- Calendar and study planning . . . 7
- Focus, offline behavior, and reminders . . . 7
- Class workspaces and tutor . . . 7
- Student control, accessibility, and quality . . . 8

Phase A - Mobile Implementation (Deferred) 9

- Objective . . . 9
- Mobile scope . . . 9
- Mobile acceptance checklist . . . 10
- Mobile handoff estimate and exit gate . . . 10

Remaining Production Work After Website Phase A 11

Phase B - Academic Intelligence and Integrations 11

- Approved integrations . . . 11
- Leander ISD Home Access Center assisted import . . . 11
- Academic intelligence . . . 12
- Recommended Phase B additions . . . 12

Phase C - Expansion 12

- Approved expansion . . . 12
- Recommended Phase C additions . . . 12

Scheduling and Priority Design 13

Technical Architecture and Interfaces 13

Delivered website structure . . . 13

Future production structure . . . 14

Core entities . . . 14

Service interfaces . . . 14

AI Implementation 15

Integration Behavior 15

Privacy, Safety, and Reliability 15

Delivery Roadmap and Effort 16

Test and Acceptance Plan 16

Documentation and Deployment Deliverables 16

Assumptions 17

Sources 17

Revision at a Glance

This edition updates the approved August 2026 plan after execution of the website portion of Phase A. It preserves the product principles, deterministic scheduling model, safety boundaries, and long-term architecture while separating the future Expo Go mobile implementation into its own Phase A workstream.

> Delivery outcome: a responsive, production-built Homework Helper website now runs as a private personal-PC service. It includes native SQLite persistence, explainable planning, class workspaces, focus sessions, offline queueing, exports, and a source-aware tutor with a credential-free fallback. External identity, cloud sync, native device services, and provider integrations remain explicit follow-up work.

Area	Current status	Evidence or next gate
Website application	Delivered	Responsive eight-area product shell, working local APIs, sample semester, dark mode, reduced motion
Personal-PC deployment	Delivered	`npm run build`, `npm start`, HTTP/API smoke test, native SQLite database
Scheduling and priority	Delivered	Approved 0-100 breakdown, editable 25-90 minute proposals, conflicts and overload explanations
Local study support	Delivered	Five tutor modes, class material isolation, citations, editable memory, local fallback
External services	Not configured	Real email/Google identity, cloud data, OCR, transcription, and provider OAuth require accounts and credentials
Expo Go mobile app	Deferred Phase A	Detailed implementation plan and acceptance checklist are included below

What was completed

- Created and worked on the requested Git branch, `firstUpdate`.
- Initialized a TypeScript vinext/Next.js website with a Cloudflare-compatible worker build.
- Added native SQLite for `npm start` on a personal PC and D1 support for worker deployments.
- Implemented a local owner session with an enforced 13+ confirmation and HttpOnly cookie.
- Implemented profile, timezone, preferred study hours, sleep hours, breaks, workload limits, theme, and reduced-motion controls.
- Implemented course workspaces, schedules, instructors, terms, grading notes, archives, and class-specific materials.
- Implemented assignments, milestones, commitments, effort, difficulty, confidence, importance, grade impact, completion, and actual-time reflection.
- Implemented pasted-text, photo/file, and browser voice capture with a draft-review-confirm workflow.
- Implemented day, week, and month calendar views plus ICS import/export.
- Implemented explainable priority ranking and a deterministic study-block proposal workflow that never edits the calendar silently.
- Implemented Pomodoro/custom focus timers, pause/resume, browser-local completion reminders, and reflection logging.

- Implemented cached-agenda startup, service-worker app-shell caching, and an offline mutation queue while keeping the server authoritative.
- Implemented tutor modes for hint, explain, worked example, check my work, and direct answer, with academic-integrity labeling.
- Implemented class-isolated citations, student-controlled tutor memory, a private local study coach, and an optional server-side OpenAI Responses API path.
- Implemented private insights, JSON/CSV exports, ICS export, sign-out, and complete account/data deletion.
- Added a bespoke social preview, checked-in SQLite migration, automated planning/build tests, and personal-PC deployment documentation.

What still needs to happen

- Add real email one-time-code and Google OAuth identity, recovery, rate limits, and abuse controls.
- Move from a single-PC database to hosted Postgres/Supabase with row-level security and true multi-device synchronization when public cloud use is approved.
- Add production OCR and server transcription for automatic photo and audio extraction.
- Add native calendar permissions and durable local notification scheduling in the Expo Go app.
- Add hosted object storage and class-specific vector/file search for large materials.
- Complete provider OAuth, token encryption, revocation, incremental sync, and health monitoring in Phase B.
- Perform accessibility, privacy, legal, dependency, secret-scanning, penetration, backup/restore, and disaster-recovery reviews before a public launch.
- Build and test the separate Expo Go mobile app described in the next section.

Product and UX

Summary

Homework Helper is a private homework planning and tutoring product for high-school and college students age 13+. The responsive website emphasizes setup and detailed management. The future Expo Go app emphasizes daily planning, quick capture, focus sessions, and tutoring. Both surfaces eventually share the same authenticated account and server-authoritative data.

Success requires:

- An explainable priority engine and editable study-block proposals.
- Class-specific tutoring grounded in student materials with visible citations.
- Secure, student-controlled integrations, exports, deletion, and offline resilience.
- A calm visual system with restrained priority colors, accessible contrast, clear typography, dark mode, and reduced-motion support.
- No public leaderboards, competitive ranking, advertising profile, or public performance data.

Core navigation

- Website: Dashboard, Calendar, Assignments, Classes, Tutor, Insights, Integrations, Settings.
- Mobile: Today, Calendar, Quick Add, Tutor, More.

Primary student journey

1. Create an eligible student profile and set timezone, available study time, sleep time, breaks, and daily limits.
2. Add classes, commitments, materials, and assignments manually or through a reviewable capture draft.
3. Inspect the priority explanation and request a study-block proposal.
4. Edit, remove, or move proposed blocks before accepting them.
5. Run focus sessions, complete milestones, and record actual time.
6. Ask for course-specific tutoring and verify cited material.
7. Export or delete personal data at any time.

Phase A - Website Implementation

Delivered personal-server experience

The Phase A website is a complete local-owner release. It can be installed on a Windows or Linux personal computer with Node.js 22.13 or newer, built with `npm run build`, and served on the local network with `npm start`. The compiled server binds to `0.0.0.0` and defaults to port 3000.

The personal-PC path uses `data/homework-helper.sqlite`. The worker development/deployment path uses the logical D1 binding `DB`. The repository ignores runtime databases and secret environment files.

Identity and student profile

- Local owner setup requires name, valid email format, and explicit 13+ confirmation.
- Server session tokens are random, HttpOnly, same-site cookies with 30-day expiration.

- Profile controls include timezone, preferred study window, sleep window, daily study limit, break length, theme, reduced motion, and tutor-memory status.
- The local owner mode is intentionally not presented as verified email or Google identity.

Courses, assignments, and commitments

- Courses carry name, code, color, term, instructor, schedule, grading notes, archive state, and materials.
- Assignments carry deadline, notes, estimated and remaining effort, difficulty, confidence, importance, grade impact, milestones, status, actual time, and source.
- Commitments carry fixed start/end time and recurrence metadata and are treated as hard busy time.
- The website ships with optional, editable sample data for immediate evaluation.

Capture and confirmation

- Pasted text is parsed locally into a title, class, due date, and effort draft with field-level confidence.
- Photo/file capture records no raw file; the student supplies or corrects visible text before extraction.
- Supported browsers can use built-in speech recognition for short voice capture.
- No extracted field saves until the student reviews and confirms the draft.

Calendar and study planning

- Day, week, and month views show commitments, accepted blocks, and assignment due counts.
- ICS import adds outside events as read-only busy time.
- ICS export includes commitments and accepted Homework Helper blocks.
- The scheduler ranks unfinished assignments, splits work into editable blocks, respects commitments and preferred hours, inserts breaks, enforces the daily cap, honors deadlines, and returns unscheduled overload risk.
- Proposed blocks show their reason and confidence. Students may edit times, remove blocks, recalculate, keep the calendar unchanged, or explicitly accept.

Focus, offline behavior, and reminders

- Focus sessions support 25, 45, and 60 minute timers, pause/resume, completion, and a reflection.
- Actual time reduces remaining effort and contributes to private insights.
- Browser notification permission may be used for a running focus timer only; there is no remote push or email reminder.
- The service worker caches the application shell. A cached agenda and queued mutation are browser-local resilience aids, not the authoritative data source.
- Optimistic version checks prevent one tab from silently overwriting a newer server snapshot.

Class workspaces and tutor

- Materials may be saved as notes, syllabus text, rubrics, study guides, or other pasted text.
- Tutor modes are hint, explain, worked example, check my work, and direct answer.
- The local study coach selects relevant class material, cites what it received, and provides structured next-step guidance without an external credential.
- When a server-side OpenAI key is configured, the Responses API path uses the selected class materials and editable tutor memory; it falls back locally if the provider is unavailable.
- All tutor output is labeled as fallible and includes an academic-integrity reminder.

Student control, accessibility, and quality

- Students may view, edit, add, disable, or delete tutor-memory items.
- Students may export JSON, CSV, and ICS files or permanently delete the account and all stored data.
- Priority is never communicated by color alone.
- The interface supports keyboard focus, semantic labels, responsive mobile-web navigation, dark mode, reduced motion, and accessible contrast targets.
- The production build and five automated tests pass. Local development and compiled personal-server API smoke tests pass.

Phase A - Mobile Implementation (Deferred)

Objective

Build an Expo Router React Native companion that runs completely inside the stable Expo SDK supported by the current Expo Go release. It shares the future production account, API, scheduling rules, domain types, and design tokens with the website. This work is intentionally deferred and must not block use of the completed personal-server website.

Mobile scope

Foundation and shared packages

- Create `apps/mobile` with Expo Router, TypeScript, and Expo-Go-compatible dependencies only.
- Extract shared domain types, validation, API client, priority calculations, schedule explanations, and design tokens into versioned packages.
- Keep platform-specific controls in React Native; do not wrap or force web DOM components into the app.
- Centralize authored English strings and accessibility labels for future localization.

Navigation and onboarding

- Tabs: Today, Calendar, Quick Add, Tutor, More.
- Add loading, signed-out, first-run, permission, offline, empty, error, and conflict states.
- Use the production email one-time-code and Google identity flow only after the backend identity service exists.
- Require the same 13+ gate and privacy explanation as the website.
- Sync profile, timezone, workload limits, preferred study time, sleep time, and availability.

Today and daily planning

- Show cached agenda immediately, then reconcile with the authoritative server.
- Show the top priority tasks with the same visible score breakdown and non-color indicators.
- Support preview/edit/accept study-block proposals and explain unscheduled overload.
- Allow complete, pause, reschedule, add note, and start focus actions with optimistic UI and idempotency keys.
- Never change accepted calendar blocks without an explicit student action.

Calendar and local reminders

- Use Expo Calendar for device busy-time access after permission.
- Import provider or device events as read-only busy time.
- Write only accepted Homework Helper blocks to an app-owned calendar where platform support allows it.
- Use Expo Notifications for local scheduled reminders only.
- Recalculate notifications after accepted blocks, completion, deadline, or sync changes.
- Do not add remote push, reminder email, native widgets, or background behavior unavailable in Expo Go.

Quick Add and capture

- Support camera photo capture, image-library selection, pasted text, and short audio capture.
- Upload to temporary server storage through signed URLs.
- Use server OCR/transcription and structured extraction with field confidence and provenance.
- Show a required review sheet for class, title, deadline, effort, difficulty, importance, and grade impact.

- Delete temporary photo/audio after confirmation or cancellation unless explicitly kept as class material.

Focus sessions

- Support Pomodoro and custom timers, pause/resume, milestones, completion, and actual-time reflection.
- Preserve timer state across navigation and ordinary app suspension within Expo Go limitations.
- Schedule a local end-of-session reminder and cancel or replace it when the timer changes.

Tutor and materials

- Reuse the five tutor modes and selected-class boundary.
- Stream answers when online and show citations as tappable source sheets.
- Show cached thread history offline but require a connection for new model responses.
- Support tutor-memory view, edit, forget, and disable controls.
- Preserve the AI label, citation warnings, and academic-integrity language.

Offline synchronization

- Cache the current agenda, course list, accepted blocks, and recent tutor history.
- Queue completion and note edits with idempotency keys and record versions.
- Replay in order, detect version conflicts, and present a student-readable resolution screen.
- A remote deletion always wins; queued edits must never resurrect deleted records.
- Keep server timestamps and the account/device timezone explicit at synchronization boundaries.

Mobile acceptance checklist

- Run every acceptance flow in current iOS and Android Expo Go releases.
- Verify Google sign-in after the production identity service is available.
- Verify camera, microphone, image library, device calendar, and local notification permissions.
- Verify launch from cached agenda, airplane-mode edits, replay, duplicate prevention, and remote deletion conflicts.
- Verify DST, timezone change, all-day events, deadlines without times, recurrence exceptions, and simultaneous edits.
- Verify screen readers, dynamic type, minimum touch targets, focus order, contrast, reduced motion, and non-color priority indicators.
- Meet targets: cached agenda appears immediately; normal sync is visible within two seconds on a typical connection; a semester-sized proposal returns within five seconds; tutor streaming visibly begins within five seconds when healthy.

Mobile handoff estimate and exit gate

Plan 12-18 person-weeks for the Expo Go Phase A app after shared APIs and production identity exist, approximately 10-14 calendar weeks for one Expo-focused engineer with design and QA support. Exit requires a deployable Expo Go build, current iOS/Android checklists, migration notes, seeded demo data, privacy review, and a go/no-go record.

Remaining Production Work After Website Phase A

Capability	Website behavior now	Production follow-up
Identity	Local owner session, 13+ confirmation	Verified email OTP, Google OAuth, recovery, abuse protection, revocation
Data	Native SQLite on one PC; D1 worker path	Hosted Postgres/Supabase, row-level security, encrypted backups, multi-device sync
Capture	Local text parser, browser speech, manual photo review	Server OCR, transcription, structured outputs, temporary blob deletion
Calendar	Internal calendar and ICS import/export	Expo device calendar plus Google/Microsoft incremental OAuth sync
Reminders	Running browser focus completion	Reschedulable native local notifications in Expo Go
Materials	Pasted text stored per class	Signed uploads, object storage, antivirus checks, class-specific vector/file search
Tutor	Local fallback; optional Responses API	Streaming, moderation, rate limits, evaluations, cost controls, hosted citations
Offline	Cached shell/agenda and one queued snapshot	Ordered mutation log, idempotent replay, conflict UI, telemetry
Operations	Build/test/deployment guide	HTTPS, monitoring, alerts, backup drills, incident response, legal/security review

Phase B - Academic Intelligence and Integrations

Approved integrations

- Google Calendar and Microsoft Graph connections initiated from the setup website.
- Import outside events as read-only busy time.
- Create or update only Homework Helper study blocks in a dedicated provider calendar.
- Preserve provider IDs and incremental-sync cursors to prevent duplicates.
- Google Classroom read-only import of courses, coursework, due dates, attachments, submission state, and grades available to the signed-in student.
- Never submit work, edit Classroom records, or change grades.

Leander ISD Home Access Center assisted import

- Open HAC in the system browser.
- Accept a student-provided printout, PDF, or screenshot.
- Extract grades/classwork into a review grid.
- Save nothing until confirmation.
- Never request or store HAC credentials.
- Delete raw HAC captures after confirmed import or cancellation.

Academic intelligence

- Syllabus onboarding proposes course details, grading weights, key dates, policies, and materials for confirmation.
- Editable decomposition of essays, projects, and exam preparation into milestones, dependencies, and effort estimates.
- Weighted gradebook, course standing, assignment-impact calculations, and clearly labeled what-if forecasts.
- Source-linked study kits containing summaries, flashcards, quizzes, explanations, and review schedules.
- Completion trends, planned-versus-actual time, overload warnings, study-time patterns, and course workload insights.
- No competitive ranking or public performance data.

Recommended Phase B additions

- Integration health center: show last successful sync, revoked/expired access, cursor age, duplicate prevention, provider outages, and guided repair.
- Student inbox: collect changed deadlines, newly imported work, sync conflicts, and low-confidence extractions into one review queue instead of silently mutating plans.
- Scenario planner: compare two schedule proposals or what-if workload changes before committing either one.
- Source coverage meter: identify tutor answers or study kits that lack enough class evidence and invite the student to add a missing source.
- Spaced-review engine: turn confirmed flashcards and quiz misses into an editable review cadence tied to exam dates.
- Accessibility personalization: save reading density, font scale, motion, color, focus-block length, and notification preferences across devices.
- Encrypted backup export and restore: include a portable manifest, version, checksum, and a dry-run validation before import.
- Integration cost and quota guardrails: expose student-safe status without exposing tokens, and pause generators when budgets or provider limits are reached.

Phase C - Expansion

Approved expansion

- Invitation-based group projects with shared milestones, task ownership, deadlines, notes, and an activity log.
- Official HAC synchronization only if Leander ISD or its student-information-system vendor supplies an approved API and agreement.
- Pilot-oriented school administration or guardian functionality only as a separately approved product expansion with legal review.

Recommended Phase C additions

- Private study rooms: invitation-only sessions with a shared agenda, timer, source board, role-based edits, and a complete activity history.
- One-way progress share: student-generated, expiring read-only summaries for an advisor or counselor; no ambient surveillance and no default guardian access.
- Standards-based school interoperability: evaluate OneRoster or LTI only through approved district/vendor agreements and least-privilege scopes.
- Privacy-preserving school insights: aggregate only with minimum cohort sizes, suppression rules, and no access to student prompts or tutor conversations.

- Advanced native companion track: if the product graduates from Expo Go, separately evaluate widgets, share extensions, biometric app locks, and remote push. None are added to the Expo Go baseline.
- On-device study tools: evaluate offline search, lightweight flashcard generation, or local text classification when device capability and privacy review support it.
- Collaboration safety: reporting, invitation limits, block controls, audit retention, and clear ownership transfer before enabling group projects broadly.

Scheduling and Priority Design

Use deterministic planning logic for trust and testability. AI may estimate missing metadata, but every estimated value is labeled and confirmed before affecting a schedule.

Calculate a 0-100 priority score with a visible breakdown:

- Deadline urgency and remaining slack: 35 points.
- Grade or outcome impact: 20 points.
- Student-set importance or consequence: 15 points.
- Remaining effort versus available time: 15 points.
- Difficulty and low confidence: 10 points.
- Dependency blocking: 5 points.

Overdue work receives maximum urgency. Missing grade data falls back to the student's low, medium, or high impact label rather than inventing a numeric grade.

The scheduler will:

1. Expand commitments and rotation rules into occupied intervals.
2. Apply sleep, no-study, break, maximum-workload, and preferred-time rules.
3. Split assignments into editable 25-90 minute blocks.
4. Honor milestone dependencies and deadlines.
5. Minimize missed deadlines first, then risk-weighted lateness, context switching, and undesirable study times.
6. Return unscheduled work and an overload explanation when hard constraints make the plan impossible.
7. Present reasons, conflicts, and confidence before the student accepts the proposal.

Handle timezones, daylight-saving changes, all-day events, recurrence exceptions, deleted provider events, simultaneous edits, and deadlines without explicit times.

Technical Architecture and Interfaces

Delivered website structure

- TypeScript vinext/Next.js App Router website with React 19.
- Cloudflare-compatible ESM worker output.
- Logical D1 binding for worker development/deployment.
- Native Node SQLite adapter for the compiled personal-PC server.
- Versioned, per-user JSON workspace snapshot with optimistic concurrency.
- HttpOnly session cookie and server-side authorization checks on all personal data routes.

- Service worker for shell caching; browser storage only for a cache and offline queue.
- Checked-in migration, seeded demo data, automated tests, and deployment handoff.

Future production structure

Use a TypeScript pnpm/Turborepo monorepo when the mobile and hosted services begin:

- Next.js/vinext website for setup and detailed management.
- Expo Router React Native app restricted to Expo-Go-compatible dependencies.
- Shared packages for domain types, Zod validation, API client, scheduling logic, priority calculations, and design tokens.
- Platform-specific UI components built on shared tokens.
- Supabase for hosted Postgres, row-level security, authentication, storage, realtime invalidation, and scheduled/edge functions after cloud approval.
- OpenAI and integration secrets remain server-side. Environment-specific identifiers remain outside source control.

Core entities

Define shared schemas for:

- UserProfile, AvailabilityRule, Course, CourseScheduleRule.
- Assignment, Milestone, Commitment, StudyBlock, FocusSession.
- GradeCategory, GradeItem, GradeForecast.
- CourseMaterial, TutorThread, TutorMemory, StudyArtifact.
- CalendarConnection, ExternalEvent, IntegrationConnection, SyncCursor.
- CaptureDraft, ScheduleProposal, PriorityBreakdown, OfflineMutation.

Every imported record carries source, externalId, sourceUpdatedAt, extraction confidence, and field-level provenance. Every mutable record carries a version for optimistic concurrency.

Service interfaces

Maintain versioned, authenticated endpoints for:

- Assignment, course, and commitment CRUD plus bulk imports.
- `POST /captures/extract` returns a structured draft with confidence and provenance.
- `POST /plans/propose` returns priority breakdown, blocks, conflicts, and unscheduled risks.
- `POST /plans/{id}/accept` atomically commits the edited proposal.
- `POST /tutor/respond` streams answer, tutor mode, citations, and optional memory proposal.
- Study-kit and milestone-generation jobs.
- Grade import, forecasting, and confirmed HAC parsing.
- Calendar/Classroom OAuth, sync, disconnect, token revocation, and health status.
- Export and account-deletion jobs.

All production mutation requests use idempotency keys. Integration tokens are encrypted, minimally scoped, revocable, and never returned to clients.

AI Implementation

- Use the OpenAI Responses API through server functions.
- Keep model IDs in configuration and gate model/prompt changes with evaluations.
- Use class-specific file search or vector stores for hosted grounded tutoring and source annotations.
- Use structured outputs for capture, syllabus parsing, milestones, and study kits.
- Transcribe short voice captures server-side, then process them like text drafts.
- Send a privacy-preserving safety identifier, moderate inputs/uploads where appropriate, and rate-limit requests.
- Prevent one class or student's files from appearing in another context.
- Delete hosted files and vector records when materials, classes, or accounts are deleted.
- Preserve the credential-free local tutor as a resilience mode, clearly labeled and never misrepresented as model output.

Integration Behavior

- Google Classroom maps courses and CourseWork into local read-only source records.
- Google Calendar uses incremental sync tokens and performs a new full sync when a token is invalidated.
- Microsoft calendars use Graph calendar-view delta links and least-privilege delegated permissions.
- Device calendars use Expo Calendar; unsupported cloud providers use device access or ICS.
- HAC remains assisted import unless an approved public student integration API and agreement exist.

Privacy, Safety, and Reliability

- Enforce row-level security in hosted production so students can access only their records and explicitly shared Phase C projects.
- Exclude under-13 registration from v1 and obtain legal review before any school-managed or guardian product.
- Encrypt data in transit and at rest. Redact prompts, grades, files, OAuth tokens, and assignment text from operational logs.
- Automatically delete temporary photo, audio, syllabus, and HAC extraction files unless explicitly kept as class material.
- Provide per-integration disconnect/revoke, material deletion, tutor-memory management, export, and account deletion.
- Treat AI citations and grade forecasts as fallible; show sources, calculation inputs, and correction controls.
- Meet WCAG 2.2 AA on the website and comparable mobile accessibility requirements.
- Record operational metrics for sync failure, scheduler failure, AI latency/cost, citation coverage, and error rate without student content.
- Keep remote push, native widgets, biometric locks, parent/teacher roles, and payments outside the approved baseline.

Delivery Roadmap and Effort

Workstream	Status	Planning range
Foundation and website Phase A	Delivered in this repository	Completed implementation; external production review remains
Expo Go mobile Phase A	Deferred	12-18 person-weeks; about 10-14 calendar weeks after identity/API readiness
Phase B intelligence and integrations	Planned	16-24 person-weeks; about 8-12 additional weeks
Phase C collaboration and partnerships	Planned	8-14 person-weeks plus approval lead time

Add contingency for Google OAuth verification, school-account restrictions, provider review, security remediation, and any district integration process. Each phase ends with a deployable release, migration scripts, seeded demo data, updated documentation, backup/restore proof, and a go/no-go checklist. Use feature flags for integrations and generators.

Test and Acceptance Plan

- Unit-test priority weights, slack calculations, grade forecasts, recurrence expansion, timezone/DST behavior, milestone dependencies, and overload detection.
- Property-test that accepted blocks never overlap hard commitments, violate sleep/no-study rules, or exceed configured limits.
- Integration-test Google/Microsoft incremental sync, revoked tokens, quotas, duplicate events, provider deletion, and full-resync recovery.
- Verify Classroom imports are student-authorized and no write operation is possible.
- Test capture with poor images, missing fields, contradictory dates, and low confidence; nothing saves before confirmation.
- Evaluate tutoring for citation correctness, class isolation, hallucination resistance, academic-integrity labeling, harmful-content handling, and memory deletion.
- Verify row-level security, signed uploads, account deletion, hosted file/vector deletion, OAuth revocation, and log redaction.
- Test offline agenda startup, queued edits, idempotent replay, remote deletion conflicts, and recovery after failed sync.
- Run website end-to-end tests for onboarding, setup, planning, focus, tutoring, exports, deletion, and integrations.
- Run the complete mobile checklist in current iOS and Android Expo Go releases.
- Perform accessibility, privacy, dependency, secret-scanning, and backup/restore audits before each public release.

Documentation and Deployment Deliverables

This update produces:

- Canonical Markdown plan: `docs/Homework-Helper-Product-and-Implementation-Plan.md`.
- Revised rendered PDF under `output/pdf/`.
- Product overview and remaining production gaps in `README.md`.
- Personal-PC server instructions in `DEPLOYMENT.md`.
- Checked-in D1/SQLite migration under `drizzle/`.

- Optional AI environment template in `.env.example`.

For a private LAN deployment, install dependencies, run the production build, start the server, allow only the selected private-network firewall rule, and back up the SQLite database. For access outside a trusted LAN, use HTTPS behind a reverse proxy plus VPN, allowlist, or another independent access control. Do not forward the raw Node port directly from a home router.

Assumptions

- English is the only authored language in v1; strings should move into localization resources during the shared-package/mobile work.
- The device/account timezone is authoritative; no timezone is hardcoded.
- External events remain read-only, and only app-owned study blocks are written to connected calendars.
- Google Classroom remains read-only, and HAC credentials are never collected.
- Smart reminders remain local-only for the Expo Go baseline.
- AI models, provider limits, SDK versions, and OAuth requirements are revalidated at implementation kickoff and pinned in configuration or lockfiles.
- Effort ranges are planning estimates, not fixed-price or revenue projections.
- The personal-PC owner controls physical access, operating-system accounts, firewall rules, backups, and TLS termination.

Sources

- OpenAI API models and Responses API availability: OpenAI Models
- Google Calendar synchronization: Google Calendar Sync
- Google Classroom resources: Google Classroom API
- Microsoft event delta: Microsoft Graph event delta
- Expo Calendar: Expo Calendar
- Expo Go limitations: Expo Go
- Leander ISD Home Access Center: Home Access Center